CS 165 — Lab 2: Controlled Buffer Overflow

This handout explains **what each step does and why**. The exact commands for each step are on the lecture slides, so they are not repeated here. One command that you will rely on — x/16gx note — is explained in full in Step 4.

The Problem

vault is a small, intentionally vulnerable program. It reads standard input in two phases: first a one-line **name** that it greets you with, then a 2-byte length followed by a binary **request** that it checks. It contains **two deliberate bugs** and it is compiled with the **stack canary turned on**. A harmless function win() prints a build-specific token (e.g. LAB2-WIN:vault-xxxxxxxx) but is never called. Your goal is to make the program reach win() anyway — which means getting past the canary — and then to close both bugs.

Bug #1 (format string): the name you supply is eventually used as a printf format string, so you can place specifiers like %p in it and make the program print values off its own stack — including the canary.

Bug #2 (buffer overflow): the request is copied into a small fixed buffer with no length check, so a long request runs past the buffer over the canary, the saved frame pointer, and the saved return address.

Why both bugs are needed. The canary is a random value regenerated on every run, so an overflow that blindly overwrites it is detected and the program aborts **before** it can return. The only way through is to **read the live canary** with Bug #1 and write that exact value back into its slot as part of the overflow from Bug #2, so the check still passes. One bug leaks the secret the other one needs.

Your instance is unique. You generate your own copy of vault.c from your student ID. Your buffer size, the position of the canary in the leak, and your token are all seeded from that ID, so your numbers differ from everyone else's. A classmate's finished exploit will not work on your instance.

The Steps

Step 1 — Generate and build your instance

Run the generator with your student ID. It writes vault.c, a Makefile, and a tests/ folder into the current directory, all seeded from your ID. Build the program, then run the benign test: the program should greet you and then accept a well-formed request and reject a malformed one. This is the baseline — the program behaving normally, before any attack. Note the build ID it prints; if you later edit the source, rebuild so the binary matches what you are analysing.

Step 2 — Find the address of win()

You will redirect execution into win(), so you need its address. Because the program is built non-PIE, win() sits at the **same fixed address on every run**, so you can read it once from the binary in the debugger — you do not even need to run the program. Record this address; in the exploit it is the value WIN. (If this were a PIE build the address would be randomized each run and you would have to leak it too, but it is not.)

Step 3 — Find the canary's leak position

The program echoes your name through the format-string bug, so sending a positional specifier prints the value at a chosen slot on the stack. Sweep the slot number until one of them prints the canary. You recognise the canary by three traits together:

- it is an 8-byte value whose **last byte is 00** (a deliberate property of the Linux stack canary);
- the other seven bytes look random;

- it **changes on every run** — confirm this by leaking the same slot a few times and watching the value differ.

Everything else is something else: 0x7fff… values are stack addresses, 0x40… values are code addresses, and anything that stays the same across runs is a constant, not the canary. Record the winning slot number as LEAK_INDEX.

Important distinction. The slot number is a fixed property of your build — it does not change from run to run, so you find it once. But the canary **value** at that slot changes every run. That is why you cannot simply read the canary in the debugger and hard-code it: by the time your exploit (or the grader) runs, it is a fresh process with a different canary. The value has to be read live, which is exactly what this leak does.

Step 4 — Find the two offsets in the live frame

You need two distances, both measured from the start of the buffer note: the distance to the **canary slot**, and the distance to the **saved return address**. Set a breakpoint inside the request-checking function and run with the benign test input so that execution reaches it with the frame fully set up. Step just past the copy so the buffer holds recognisable bytes, then examine the stack starting at the buffer and read off the three landmarks.

Examine the stack with this command (it is the one command worth spelling out here):

```
(gdb) x/16gx note
```

It means: **examine 16 giant (8-byte) words, in hexadecimal, starting at** note**.** The pieces are:

- x — examine memory (read raw bytes at an address, as opposed to print, which evaluates an expression);
- 16 — how many units to show;
- g — the unit size: "giant" word = 8 bytes, the size of a pointer, the canary, and the return address, so each appears as one clean word;
- x — the format: hexadecimal;
- note — the start address. An array name evaluates to its own address, so this starts at the buffer.

So it reads 16 × 8 = 128 bytes beginning at the buffer — enough to cover the buffer plus the canary, the saved frame pointer, and the saved return address above it.

Step 5 — Complete exploit.py

The script already contains the plumbing that launches the program, sends the leak, reads the canary back, and sends the final request. You supply two things: the four values you measured (WIN, LEAK_INDEX, OFF_CANARY, OFF_RETADDR), and the body of the payload-building function.

The payload is laid out to match the frame, low address to high:

```
[ filler ][ canary ][ saved rbp ][ return address ]
```

so the function must: fill up to the canary slot; write the **leaked** canary there so the check passes; fill the gap over the saved frame pointer; and write WIN over the saved return address. A few points to understand, which belong in your writeup:

- Only the filler **length** matters, not its contents — it exists solely to position the next bytes.
- The canary must be written back **unchanged**; a single wrong byte trips the check.
- Addresses are written in **little-endian** order (the helper in the script does this for you).
- The copy uses memcpy, which copies raw bytes including the canary's 00 byte; a string copy would stop at that 00 and fail.

Step 6 — Run the exploit

Run the script. In one process it sends the leak, reads your live canary, builds the payload, and sends the overflow. Success prints your LAB2-WIN token and a note that it reached win(). Read failures like a diagnosis:

- **"stack smashing detected" / abort** — the canary in your payload was wrong (bad LEAK_INDEX, or you did not write the leaked value back correctly).
- **a plain crash with no token** — an offset or WIN is wrong, so you reached the return but jumped somewhere invalid.
- **a segfault AFTER the token prints** — this is expected and harmless: win() returns into filler once it has already printed the token.

Step 7 — Fix both bugs

Make a minimal source change for each bug.

Step 8 — Write the report (writeup.md)

About one page, in your own words:

- the two bugs and why each is dangerous;
- your four values and **how you found them** (include a short debugger transcript);
- why leaking and replaying the canary defeats the stack protector here;
- the source fix you made for each bug, and why it closes it;

What to Submit

Submit exactly three files:

- exploit.py — your completed exploit (reaches win() on a freshly generated copy of your instance).
- vault.c — your **fixed** source: both bugs closed, benign requests still work.
- writeup.md — your report.

Rules & Scope

vault is a local teaching target. Do **not** attempt to spawn a shell, use sudo, disable VM protections system-wide, or run any of this against another program. Work only on the course VM. The aim is to understand the attack well enough to **defend** against it.